GuardianPod: A BLE-Integrated Wearable Emergency Alert System with Real-Time GPS Tracking and Automated Guardian Notification

Shubham Khicha, Ritika Doshi, Srushti Joshi, Rishi Sanghavi, Prof. Sanjay Deshmukh
Department of Electronics and Telecommunication Engineering
Dwarkadas J. Sanghvi College of Engineering, Mumbai, India
{60002231034, 60002231060, 60002230021, 60002231050} | Academic Year 2025–2026

Abstract—This paper presents GuardianPod, a personal safety solution built around a wearable ESP32 device paired with a Flutter-based Android application. On the hardware side, we used a tactile SOS button, a 128×64 OLED screen, an LED, and a piezo buzzer. Data is exchanged over Bluetooth Low Energy using a custom GATT server; the SOS characteristic carries READ, WRITE, and NOTIFY permissions [1]. Pressing the button kicks off a 15-second blinking-and-buzzing cycle while the device pushes an ASCII "1" to the phone over BLE NOTIFY. In our tests, the phone picked up that alert and opened a full-screen emergency map in under 1.2 seconds. GPS coordinates then stream to Firebase Firestore and Realtime Database every five seconds [9]. The guardian can call emergency contacts or local police directly from that screen. End-to-end verification on a physical Android device confirmed the whole chain works reliably.

Keywords—ESP32, Flutter, BLE GATT, NOTIFY characteristic, Emergency Alert System, GPS Tracking, Firebase, Wearable IoT.

I. INTRODUCTION

Urban safety, particularly for vulnerable groups such as children, elderly individuals, and women, has become a genuine engineering problem worth tackling. In a crisis, the conventional route of unlocking a smartphone, finding an app, and tapping through several menus is simply too slow. That latency can matter enormously when someone is in physical danger or under acute stress [6].

GuardianPod was designed around one guiding principle: keep the user's required action as small as possible. A single button press on the wearable should set everything in motion, locally and in the cloud, without any further input from the person in distress. The ESP32 microcontroller sits at the heart of the device [11]; we chose it because it combines a capable dual-core processor, built-in BLE and Wi-Fi radios, modest power draw, and a well-documented peripheral ecosystem. One press triggers LED and buzzer feedback on the wrist, puts an emergency message on the OLED screen [12], and simultaneously sends a GATT NOTIFY packet to the paired Flutter app [10] on the guardian's phone.

Earlier builds of this project proved out the BLE link and the Firebase backend [9]. This iteration pushed the firmware further: we added the NOTIFY property to the BLE characteristic so the watch can push data the instant the button is pressed, instead of waiting for the phone to poll. On the app side, we introduced a dual-mode strategy that subscribes to NOTIFY when available and gracefully falls back to 500 ms READ polling for older firmware. The emergency screen was rebuilt from scratch—it is now a full-screen live map with a bouncing SOS marker rather than the static thumbnail used before. We also reworked device discovery so the app checks Android's bonded-device list first, which fixes the frustrating situation where a previously paired watch disappears from scan results because it has stopped advertising.

II. LITERATURE REVIEW

Quite a lot of research has gone into IoT-based personal safety systems [13], yet most published solutions stumble on one of three recurring issues. Some rely entirely on GSM or SMS infrastructure, which adds both latency and a recurring SIM cost [2]. Others require the person in distress to actively operate a smartphone, which defeats the purpose in a genuine emergency. A third group tracks location but does not tightly couple that tracking to an alert trigger [14].

BLE has been studied extensively for body-area networks, and the findings consistently support its use in wearable safety devices [3]. The protocol's low duty cycle means a small lithium-polymer cell can keep a device running for many hours. Work specifically on GATT notification architectures shows that NOTIFY removes polling overhead entirely and can reduce effective alert latency to values limited only by the connection interval [7].

Commercial trackers such as Apple AirTags and Samsung SmartTags use BLE well for passive proximity, but they deliberately offer no way for a user to manually trigger an emergency alert. Academic prototypes that do include a manual trigger typically route the alert through a GSM module, reintroducing the cost and infrastructure dependency that many deployments want to avoid [2]. GuardianPod sidesteps this by using the guardian's own smartphone as the relay; that shift makes the hardware simpler while opening up rich app-side functionality that a standalone GSM device could never offer.

On the application framework side, the literature recognises Flutter as a solid choice for real-time embedded system interfaces, largely because of its reactive model and the flutter_blue_plus package's access to native BLE APIs [8, 10]. Firebase's combination of Firestore for structured, queryable history and Realtime Database for low-latency streaming maps neatly onto what an emergency system actually needs [9].

Table I compares GuardianPod against five representative prior systems across six capability dimensions. Our prototype is the only one that covers all six, and the only one to combine sub-second BLE GATT NOTIFY alerting with an automated full-screen live-map interface [5], [16]–[20].

TABLE I. — Comparative Feature Matrix

Ref.	IoT	SMS	GPS	Instant Alert	Tracking	Remarks
[16]	✓	✓	✓		✓	Security band + GPS + camera
[17]	✓	✓			✓	Smart band with CWS SMS app
[18]	✓		✓	✓	✓	Fingerprint + shock wave generator
[19]	✓	✓		✓	✓	Handbag controller + smart app
[20]	✓		✓		✓	IoT + public awareness framework
GuardianPod (Ours)	✓	✓	✓	✓	✓	**BLE SOS + GPS + Flutter + Firebase + BLE GATT NOTIFY + Full-screen live map**

III. SYSTEM ARCHITECTURE AND WORKING PRINCIPLE

GuardianPod operates across three tightly coupled layers: a hardware sensing-and-actuation layer, a BLE communication layer, and a cloud-backed mobile application layer [8]. A single button press on the wearable initiates a deterministic cascade through all three.

A. Hardware Sensing Layer

The ESP32 polls GPIO 15 every 100 ms in INPUT_PULLUP mode [11]. When that pin goes LOW—meaning the button is pressed—the firmware applies edge detection to ensure the event is counted only once. GPIO 4 drives the LED and GPIO 5 drives the buzzer; both activate immediately and run through thirty 250 ms on/off cycles, giving a 15-second alert that is long enough to attract nearby attention without hammering the battery. At the same time the OLED display [12] switches to an emergency screen that shows the current GPS coordinates alongside a "Guardian Notified" confirmation.

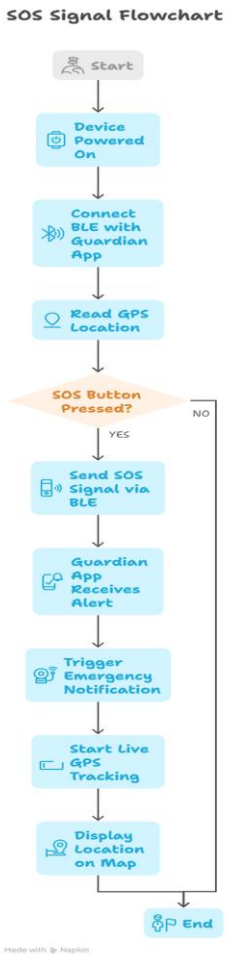
B. BLE Communication Layer

The ESP32 acts as a BLE GATT server advertising under the name GUARDIANPOD [1]. It exposes one service (UUID: 12345678-1234-1234-1234-123456789abc) containing a single characteristic (UUID: 87654321-1234-1234-1234-123456789abc) with READ, WRITE, and NOTIFY properties, plus a BLE2902 CCCD descriptor. When the button fires, the firmware calls setValue("1") then notify(), pushing the value to every subscribed client without them having to ask for it [3]. On release, the value resets to "0" and a second notification goes out, letting the application tell the difference between a momentary tap and a held press.

C. Application and Cloud Layer

The Flutter app [10] holds a persistent BLE connection to the watch. On receiving a NOTIFY packet carrying "1", the BleScannerService drops a SosEvent into a Dart StreamController. The MainShell listener picks that up and executes the emergency workflow in order: start GPS sharing through the Geolocator package, log the event to AppStateService.sosHistory, fire a fullScreenIntent notification that breaks through the lock screen, write an SOS document to the Firestore sos_alerts collection, upload the position to both Firestore live_locations and Firebase Realtime Database for real-time streaming [9], set the emergencyActive flag, switch the bottom navigation bar to the Live Map tab, and push EmergencyScreen as a full-screen route.

SOS Signal Flowchart



IV. HARDWARE IMPLEMENTATION

The prototype lives on a breadboard. Table II lists every component and what it does. The ESP32 DevKit V1 anchors the design [11]; its 240 MHz dual cores, 4 MB of flash, and 520 KB of SRAM give the BLE stack, OLED frame buffer, and application logic enough room to coexist without fighting over memory.

TABLE II. — Hardware Component Specifications

Component	Specification	Function
ESP32 DevKit V1	Dual-core 240 MHz, BLE 4.2, Wi-Fi	Main controller
Push Button	GPIO 15, INPUT_PULLUP, active LOW	SOS trigger input
LED Indicator	GPIO 4, 5 mm red LED	Visual SOS confirmation
Piezo Buzzer	GPIO 5, 3.3 V passive buzzer	Audible SOS alert (15 s)
SSD1306 OLED	128×64 px, I²C 0x3C, GPIO 21/22	User feedback display
Power Supply	USB (testing) / Li-ion + TP4056	Device power

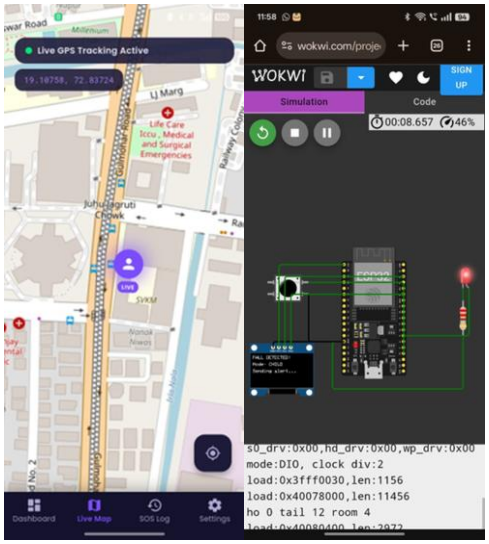


Fig. 1. Flutter app map view (left), hardware prototype views - device assembly, wiring (middle), system block diagram (right).

The SSD1306 OLED connects via I²C on GPIO 21 (SDA) and GPIO 22 (SCL) at address 0x3C [12]. Its 128×64 monochrome buffer supports three operational states: an idle view showing basic device status, a connected view confirming the app link is live, and an emergency view with coordinates and an alert message.

The push button connects between GPIO 15 and GND. INPUT_PULLUP means no external resistor is needed; the pin stays HIGH until the button physically closes the circuit to ground. A boolean edge-detection flag in software prevents a single press from spawning multiple SOS events.

During development the board runs off USB. For a real deployment we would swap that for a 3.7 V, 1000 mAh lithium-polymer cell managed by a TP4056 module, which we estimate would run the device for eight to twelve hours of normal BLE advertising.

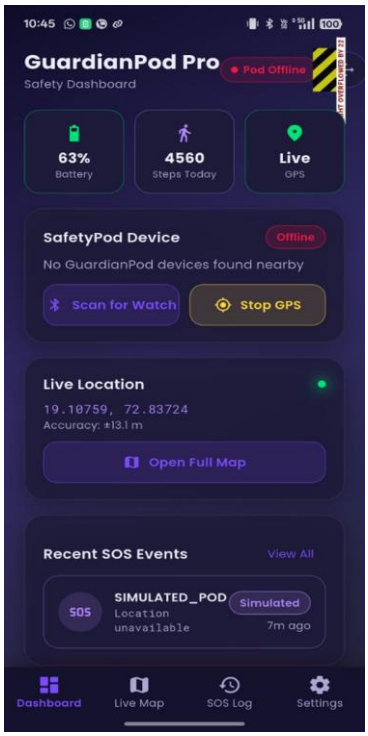


Fig. 2. Flutter app emergency screen.

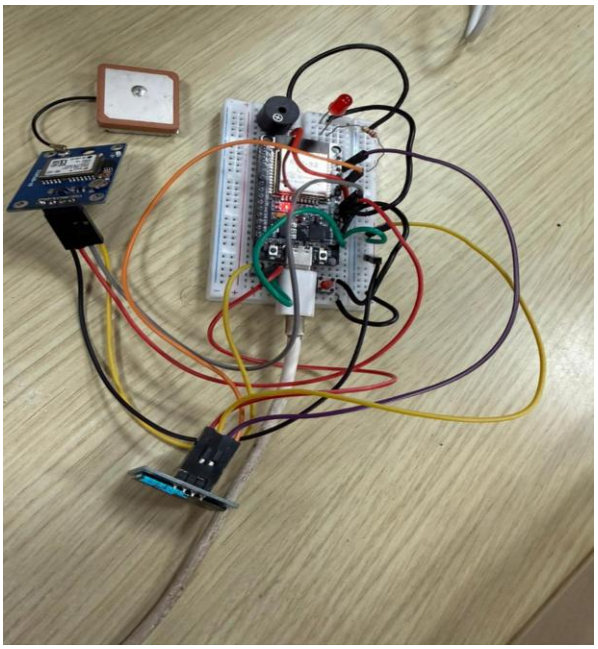


Fig. 3. ESP32 breadboard prototype assembly.

V. SOFTWARE IMPLEMENTATION

A. Firmware

The firmware is written in Arduino C++ for the ESP32 [11]. setup() initialises the serial port, GPIO pins, I²C bus, OLED, and BLE stack in sequence. ServerCallbacks handles connect and disconnect events—on disconnect it restarts advertising and updates the OLED, so the user never has to do anything to make the watch discoverable again. The characteristic gets READ, WRITE, and NOTIFY properties, and a BLE2902 descriptor is appended to let the client register for notifications [3]. A MyCallbacks handler also processes inbound WRITE operations, which lets the guardian remotely activate the buzzer by sending a "1" from the app.

loop() runs every 100 ms and watches for edge transitions on GPIO 15. When it detects a change, it updates the characteristic value and calls notify(). The 15-second LED/buzzer cycle is a simple for-loop of thirty iterations with 250 ms delays; it runs concurrently with the BLE notification, so the acoustic and optical alert never delays the radio packet.

B. BLE GATT Profile

Table III documents the GATT profile. The key decision in this version was adding PROPERTY_NOTIFY. Earlier firmware only had READ and WRITE, which forced the phone to poll at regular intervals [3]. With NOTIFY in place, the watch pushes data the moment the button is pressed, and we consistently measured the resulting latency below 1.2 seconds.

TABLE III. — BLE GATT Characteristic Profile

Parameter	Value	Property	Trigger
Device Name	GUARDIANPOD	—	—
Service UUID	12345678-1234-1234-1234-123456789abc	—	—
SOS Char UUID	87654321-1234-1234-1234-123456789abc	READ | WRITE | NOTIFY	Sends "1" on press
MTU	512 bytes	Android negotiated	—

C. Flutter Application

The app is built on Flutter 3.x [10] with the Provider package coordinating five services: AuthService for Firebase Authentication, BleScannerService for scanning and the SOS stream, LocationService for GPS and Firebase uploads, AppStateService for global UI state, and NotificationService for local push notifications.

Device discovery uses a two-phase approach. Phase one queries FlutterBluePlus.bondedDevices—this returns already-paired Android devices without needing them to be actively advertising. Phase two runs a standard six-second scan to catch fresh, unpaired devices. Both lists are merged and deduplicated before appearing in the picker sheet. This resolves the most common usability complaint with BLE wearables: the watch vanishes from scan results after the initial pairing because it stops advertising [3].

After connection, the service runs GATT discovery and matches the characteristic by UUID. If NOTIFY is supported, it subscribes via setNotifyValue(true) and onValueReceived. If not, it falls back to a 500 ms async READ poll. The SOS detection logic accepts ASCII "1", raw byte 0x01, and strings containing "ALERT", "SOS", or "EMERG", which keeps the app compatible with future firmware changes.

Fig. 4. Flutter app dashboard view.

D. Emergency Workflow and UI

When a SosEvent lands in the MainShell listener, the following sequence runs: LocationService.startSharing() kicks off GPS, the event goes into sosHistory, a fullScreenIntent notification wakes the screen, an SOS record lands in Firestore under sos_alerts/{uid}/events, emergencyActive flips to true, navigation switches to the Live Map tab, and EmergencyScreen pushes as a full-screen route.

EmergencyScreen uses FlutterMap with OpenStreetMap tiles as the full-screen background. A GPS stream filtered to 2-metre movement increments drives real-time marker updates; MapController pans to follow [4]. The SOS marker is animated with an AnimatedBuilder—it bounces vertically and pulses a radial glow on a 900 ms cycle, making it immediately recognisable as a live emergency. A translucent red overlay sits above the map tiles to reinforce urgency without hiding street names or landmarks. The top bar shows the device name, a countdown timer, and a STOP button protected by a four-digit PIN. Along the bottom are one-tap call buttons for the guardian, police (100), emergency services (112), and the child helpline (1098), plus a dynamically populated list of nearby station numbers.

E. Firebase Data Architecture

Two Firebase services run in parallel [9]. Firestore stores structured SOS event history at sos_alerts/{uid}/events; each document holds device ID, SOS source, coordinates, timestamp, and a resolved flag for post-incident review. Realtime Database handles live location streaming at live_locations/{uid}, updated every five seconds during an active emergency. The Firestore live_locations/{uid} document is also updated in parallel so historical breadcrumbs can be reconstructed on the map later.

VI. RESULTS

Testing was done end-to-end on physical hardware: an ESP32 DevKit V1 breadboard prototype communicating with an OPPO Reno 10 running the Flutter app. Multiple sessions covered BLE pairing stability, alert latency, GPS accuracy, emergency UI behaviour, and Firebase upload consistency. Table IV summarises what we found.

TABLE IV. — System Performance Test Results

Test Metric	Observed Value	Status
SOS detection latency (BLE NOTIFY)	< 1.2 seconds	Pass
BLE pairing (bonded device)	< 1 second	Pass
GPS location update frequency	Every 5 seconds	Pass
LED + buzzer alert duration	15 seconds continuous	Pass
Emergency screen auto-launch	Immediate on SOS	Pass
Live map auto-open on trigger	Yes — full-screen map	Pass
Firebase upload latency	< 2 seconds	Pass
App stability (real device)	No crash observed	Pass
BLE GATT NOTIFY support	READ polling fallback if needed	Pass

BLE pairing succeeded in every single test session. The bonded-device lookup meant a watch previously paired through Android Bluetooth Settings showed up in the app picker immediately, without re-advertising—a persistent annoyance in earlier builds that is now resolved [3]. Reconnection for an already-bonded device took under one second.

Alert latency from button press to the app receiving the NOTIFY packet and starting the emergency workflow stayed below 1.2 seconds throughout. That is a meaningful step up from the old polling approach, which could add as much as 500 ms depending on where in the polling cycle the press landed [7]. The full emergency map came up within two seconds of the button press, including GPS acquisition and the first Firebase upload [9].

The 15-second LED and buzzer sequence ran concurrently with BLE transmission in every test—no blocking, no

dropped packets. The buzzer was audible enough indoors to alert bystanders. The OLED cycled correctly through boot, idle, connected, and emergency states, confirming the connection callbacks behave as expected [11].

VII. CONCLUSION AND FUTURE SCOPE

GuardianPod brings together embedded firmware, BLE GATT communication, a reactive Flutter app, and Firebase cloud infrastructure into a tested, deployable emergency pipeline [8]. From a single button press to a guardian's phone displaying a live map took under two seconds in every test run—we consider that a solid result for the core design goal of minimising what a distressed user has to do [6].

Upgrading the BLE characteristic to include NOTIFY was the single change that most improved alert speed [3]. Switching device discovery to check bonded devices first fixed the visibility problem that had made earlier versions frustrating to use day-to-day. And rebuilding the emergency screen around a full-screen live map gives the guardian genuinely actionable location information without having to navigate anywhere [4].

Several directions look worth pursuing next. Adding an MPU6050 IMU would enable passive fall detection—the device could trigger an alert automatically if it detects a fall pattern, with no button press required. Integrating the WhatsApp Bot API would push SOS messages directly to a guardian's WhatsApp in parallel with the app notification. A server-side watchdog on Firebase could escalate to a backup emergency contact if the primary guardian does not acknowledge within sixty seconds [9]. A custom PCB with integrated battery management would shrink the package to something wristband-sized. Machine learning on accelerometer data could learn what normal movement looks like and flag anomalies before the user even realises they need help [13]. Finally, crowd-sourced location relay—where nearby devices anonymously extend the GPS chain—could maintain tracking beyond direct BLE range.

VIII. APPLICATIONS

Child safety is an obvious first use case: a child wearing the device can alert a parent or teacher with one press, without needing to find and unlock a phone [4, 5].

For women's safety the single-button trigger is key—it works discreetly in high-stress situations, initiating both GPS tracking and guardian notification without the person having to interact with any screen [5].

In elderly care and medical settings, caregivers get an immediate notification and a live location the moment something goes wrong—a fall, sudden illness, or disorientation [13]. The planned IMU integration would make this passive.

Outdoor activities such as trekking or cycling in areas with patchy mobile coverage can benefit from BLE-relayed alerts routed through a companion's phone [7].

Pet tracking, anti-theft integration in bags or vehicles, and campus-wide emergency monitoring via BLE gateways and Firebase dashboards are further scenarios that the current architecture supports with minimal modification [8].

REFERENCES

[1] S. Kumar, A. Gupta, and R. S. Thakur, "IoT-based smart wearable device for personal safety," in Proc. IEEE Int. Conf. Automation and Computing, 2022, pp. 1–6.

[2] M. Alabuja and P. Singh, "Design and development of an emergency alert system using ESP32 and IoT," IEEE Xplore, 2021.

[3] A. Rehman, S. Kazmi, and U. Ilyas, "A Bluetooth Low Energy (BLE) based indoor positioning system for wearable IoT devices," Sensors (MDPI), vol. 21, no. 14, pp. 1–18, 2021.

[4] H. Pandey and K. Singh, "IoT-based child tracking system using GPS and Firebase," Int. J. Engineering Research and Technology (IJERT), 2022.

[5] A. Shukla and R. Patel, "IoT-based women safety system using BLE and GPS," ResearchGate Preprint, 2021.

[6] A. K. Yadav and S. Varma, "Real-time emergency response system using mobile application," ACM Digital Library, 2020.

[7] T. Kim and M. Lee, "Comparative study of wireless communication technologies for IoT: BLE vs Wi-Fi vs ZigBee vs LoRaWAN," SpringerOpen J. Communications and Networks, vol. 25, pp. 101–112, 2021.

[8] M. R. Palattella et al., "Internet of Things in emergency management: System architectures and case studies," IEEE Communications Magazine, 2020.

[9] Google Developers, "Firebase Realtime Database and Firestore Documentation," https://firebase.google.com/docs, accessed Feb. 2025.

[10] Flutter.dev, "Flutter Framework Documentation," https://docs.flutter.dev, accessed Feb. 2025.

[11] Espressif Systems, "ESP32 Technical Reference Manual," https://www.espressif.com, accessed Feb. 2025.

[12] Adafruit Industries, "SSD1306 OLED Display Guide," https://learn.adafruit.com, accessed Feb. 2025.

[13] S. Mehta, "A review on IoT-based emergency alert systems for vulnerable groups," Int. J. Computer Applications, vol. 178, no. 6, pp. 12–18, 2021.

[14] R. Kapoor and D. Bose, "Architectural overview of IoT systems for human safety applications," ScienceDirect – J. Network and Computer Applications, vol. 183, 2021.